

Section 4.5

The Heavy Ranker

Building the final ranking architecture, step by step

Meta E4 MLE interview preparation

Conceptual architecture → production reality (TransAct v2)

Sourced from: Alex Xu ML System Design, Hello Interview (incl. comment thread), Instagram Explore & Facebook Reels engineering blogs

- Part A** The conceptual architecture
- Step 5** The two input streams
- Step 6** Input processing
- Step 7** The transformer blocks (self & cross attention)
- Step 8** The multi-task heads
- Step 9** Pointwise ranking & the value model
- Step 10** The loss function
- Worked example: full conceptual forward pass
- Part B** Production reality
- Step 11** Where the conceptual design breaks
- Step 12** Evolving into TransAct
- Step 13** The full TransAct v2 flow + worked example
- Reels** The UTIS perception layer
- Q&A** Tricky questions an interviewer will drill on

Where 4.5 sits

Trace the funnel. Candidate generation took billions of videos down to thousands. The light ranker took thousands down to ~100 using cheap, cacheable features. The **heavy ranker** is the last modeling stage: it takes those ~100 candidates, scores each precisely, and produces the ordering the user sees.

WHY WE CAN SPEND COMPUTE HERE

Funnel economics. At the top, a model runs against an enormous set, so it must be cheap. By the heavy ranker, only ~100 candidates remain, so even an expensive model runs only ~100 times per request. The funnel is a **compute-budgeting strategy**: cheap models on large sets, expensive models on small sets.

The defining capability unlocked at this stage: the heavy ranker can finally consume **user–item interaction (cross) features** — signals that exist only for a specific user paired with a specific video ("how many of this creator's videos has this user finished?"). These are the most predictive features in the system. The light ranker had to give them up, because its speed trick was caching independent user and item embeddings, and a cross feature — depending on both together — cannot be baked into either cached side. The heavy ranker runs on only ~100 candidates, caches nothing, and computes cross features freshly.

SOURCING NOTE

Alex Xu's chapter is thin on the heavy ranker (its "scoring" stage only says "heavier model, more features"). The substance here comes from the **Hello Interview** write-up (model progression, transformer architecture, loss) and the **Instagram Explore** blog (multi-task framing, value model), with production detail from the Hello Interview **comment thread**.

WHAT IS SOURCE-BACKED VS. FILLED IN (VERIFIED AGAINST THE FOUR SOURCES)

This guide mixes three things, kept honest throughout:

- **Source-backed** — funnel logic, cross-feature unlock, Bad/Good/Great, multi-task heads, value-model formula, the three-part loss *terms*, pointwise ranking, the TransAct 9-step flow, causal masking + hard negatives (both explicit in the comment thread), and all UTIS facts/metrics.
- **Standard convention, NOT in the four sources** (filled in for understanding; an interviewer could press here) — that embeddings are *summed* not concatenated; the Q/K/V projections and $\sqrt{d_k}$ scaling; DLRM's internal layers; which head uses BCE vs. MSE; the DCN-v2 cross-layer formula; the randomization/propensity *procedure* behind position weighting (though the terms "inverse-propensity" and "counterfactual logging" are in the source).
- **Illustrative numbers** — every vector, weight, and activation in the worked examples is invented to show mechanics (arithmetic verified); none are from a source.

Individual spots below are also tagged where it matters.

VERIFIED AGAINST ORIGINAL PAPERS (JUNE 2026)

The three formula fill-ins were checked against the source papers, all **confirmed correct**: scaled dot-product attention $\text{softmax}(QK^T/\sqrt{d_k})V$ (Vaswani et al. 2017); DCN-v2 cross layer $x_1 = x_0 \odot (W \cdot x_1 + b) + x_0$ (Wang et al. 2021); DLRM (dense $\rightarrow$ bottom MLP, sparse $\rightarrow$ embeddings, dot-product interactions, top MLP $\rightarrow$ sigmoid; Meta 2019). The TransAct v2 flow was checked against the actual paper (arXiv 2506.02267): point-wise MTL, **weighted cross-entropy per head** (confirming the BCE fill-in), the auxiliary **Next Action Loss**, the ~ 90 th-percentile/2-year lifelong sequence, action-type/surface/timestamp + content embeddings, max-pool compression, and DCN-v2 crossing are all confirmed. Two refinements: per-item candidate **early fusion** is documented for TransAct v1 and inherited by the lineage (treat as TransAct-family technique, not asserted as a distinct v2 step); and **causal masking** is implied by the next-action task but not stated outright in the verified section.

Model selection: Bad $\rightarrow$ Good $\rightarrow$ Great

Tier	What	Limitation
Bad Simple MLP	Concatenate dense (watch time, views) + sparse (creator, content embeddings), feed a plain feed-forward net.	Misses high-order interactions; no sequence/time; hard to train under sparsity.
Good DLRM	Meta 2019. Treats sparse vs. dense differently (two internal "towers"), fuses with few MLP layers.	Treats history as an unordered bag — no sequence, can surface stale/repetitive content.
Great Transformer	Models history as a sequence ; attention learns item-to-item interactions; handles temporal dimension; learns from + and – feedback.	More expensive; sensitive to training-data prep; harder to debug; may need sparse attention / MoE to scale.

SYNTHESIS (NOT IN THE FOUR SOURCES)

DLRM internals from the paper: dense $\rightarrow$ bottom MLP; categorical $\rightarrow$ embedding lookups; explicit pairwise dot-product interactions; concatenate $\rightarrow$ top MLP. The article only says "two internal towers fused with relatively less MLP layers."

Framing the article stresses: senior candidates should know the transformer shift and bias toward it; and most matrix-factorization / collaborative-filtering material online is no longer state of the art.

Part A · The conceptual architecture

We build the architecture one layer at a time, each motivated by the problem it solves.

Step 5 · The two input streams

THE PROBLEM

A ranker predicts how a *specific user* responds to a *specific candidate*, so it needs both in front of it. But the two are not symmetric — and that asymmetry drives everything.

- **Candidate + context (fixed-size).** Video ID, creator, topic embeddings, plus time/device/current-video. One creator, one topic vector — lays out as a fixed-length vector.
- **User history (variable-length).** The sequence of watched/liked/searched/skipped videos. One user has 50 interactions, another 50,000, and **order carries meaning**.

THE HOOK

The candidate is ready to use as-is; the history is not. Almost every design choice that follows exists to deal with the history side: *how do you turn a variable-length, ordered history into something combinable with the fixed candidate?*

Step 6 · Input processing

THE PROBLEM

Raw inputs aren't numbers a network can read. A `video_id` is a category; a view count is an unscaled number; the history is a bare list with no order or action type.

Job	What it does
Embeddings	Categorical IDs (<code>video_id</code> , <code>creator_id</code>) → learned vectors via lookup. Similar videos get similar vectors.
Normalization	Numerical features rescaled to comparable ranges so a millions-scale feature doesn't swamp a 0–1 one.
Positional encoding	Stamps each history item with its position, recovering the order DLRM threw away.
Action tokens	Tags each item watched / liked / skipped, so opposite reactions aren't identical.

Worked example (dimension 2)

One history item carries three facts, each its own vector, **summed element-wise** into one same-width vector (the standard transformer convention — sum, not concatenate; *the sources name positional encoding and action tokens but don't specify summing*):

```
item 0 #4471 WATCHED (pasta)
  content [ 0.1, -0.6, 0.7, 0.2 ] (shown here at dim 4 for the three-way sum)
  position [ 0.0, 1.0, 0.0, 1.0 ] (distinct fingerprint per position)
  action  [ 0.0, 0.0, 0.1, -0.1 ]
-----
sum      [ 0.1, 0.4, 0.8, 1.1 ] one fully-encoded item
```

Every item becomes one row; the rows stack into a **matrix** (one row per item). A user with 100 items → a 100-row matrix; the width is fixed, the row count varies — that is what "variable-length" means. The candidate, being one item, is a single row.

PER-USER, NOT ONE GIANT MATRIX

The history matrix is per-user, per-request. During *training*, many users are batched, but as **separate stacked matrices** — attention stays inside each user; one user's data never mixes with another's. At serving it's just the one user.

Step 7 · The transformer blocks

THE PROBLEM

After input processing we have a history matrix and a candidate vector, but nothing has *related* them. Two relationships matter, each handled by a different attention type.

The shared mechanism (Q, K, V)

No vector is used raw. Each is multiplied by three learned matrices to produce a **Query** ("what I look for"), **Key** ("what I'm matched on"), and **Value** ("what I return"). Then:

$$\text{Attention} = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

1. `scores` = each query · each key
2. `weights` = `softmax(scores /  $\sqrt{d_k}$ )` # $\sqrt{d_k}$ keeps scores from blowing up softmax
3. `output` = weighted sum of the values

Separating "what you match on" (key) from "what you return" (value) is what makes attention expressive rather than a plain similarity lookup.

	Query from	Key & Value from	Changes per candidate?
Self-attention	the history	the history	No — same for all 100
Cross-attention	the candidate	the history	Yes — recomputed each

INTUITION

Self-attention answers "*who is this user?*" (history understands itself, candidate-independent). Cross-attention answers "*which parts of this user matter for this candidate?*" (candidate filters the history). You need both.

Step 8 · The multi-task heads

THE PROBLEM

The transformer gives a rich representation, but we need predictions. Predict *what*? One objective is a trap: clicks → clickbait; raw watch time → autoplay traps; likes → sparse. Each signal is a partial, noisy view of value.

So the model has **multiple heads**, each a small layer on the shared representation predicting one signal: watch time, CTR, like, share, completion, return-visit. (Instagram Explore's multi-task multi-label / MTML model.)

Benefit	Why
Regularization	One representation supporting six predictions can't overfit one metric's quirks.
Auxiliary signal	Dense signals (watch time) lend statistical strength to sparse ones (shares).
Feeds the value model	Separate scores can be re-weighted; one blended score couldn't.
Cold-start	Cross-task correlations help when data is thin.

Step 9 · Pointwise ranking & the value model

Six predictions, one list to show. The **value model** collapses them into one scalar:

$$\text{Value} = W_{\text{click}} \cdot P(\text{click}) + W_{\text{like}} \cdot P(\text{like}) - W_{\text{see_less}} \cdot P(\text{see less}) + \dots$$

The weights encode business priorities and are tuned **without retraining** — the payoff of keeping the heads separate (Step 8). Score each candidate independently, sort by Value.

POINTWISE LEARNING-TO-RANK

No pairwise/listwise ranking loss is needed: predicting engagement *is* ranking, since higher predicted engagement → higher position (per Stefan Mai answering "Juan" in the comments). This is **pointwise**; pairwise & listwise are alternatives that sometimes do better in IR ranking.

Step 10 · The loss function

The Hello Interview article gives a three-part loss. Remember the **structure**; the exact formulas are illustrative (the sources disagree on one term).

```
L_engage    = -Σ ( y_true · log(y_pred) · watch_time_weight )
L_aux       = α·L_click + β·L_completion + γ·L_satisfaction
L_position  = 6 · BCE(y_true, y_pred) · position_weight
L           = L_engage + L_aux + L_position
```

The building block

```
binary label → cross-entropy:  -[ y·log(p) + (1-y)·log(1-p) ]
continuous  → squared error:   ( y - p )2

predicted 0.9 for an event that happened → -log(0.9) = 0.11    (small)
predicted 0.1 for an event that happened → -log(0.1) = 2.30    (huge)
```

The log is what punishes confident-and-wrong hard while making confident-and-right nearly free.

L_engage — primary signal (worked)

Two users, both a high-quality watch (`y_true=1`), model predicted `0.8` :

```
User A — full video → weight 2.0:  -1 · log(0.8) · 2.0 = -1·(-0.22)·2.0 = 0.44
User B — 5s, bounced → weight 0.3:  -1 · log(0.8) · 0.3 = -1·(-0.22)·0.3 = 0.07
```

Same prediction and label — the `log(0.8)=-0.22` is identical. Only the weight differs, so getting User A wrong costs 6× more. The 0.8 enters *through the log*.

THE REGRESSION-VS-CLASSIFICATION MISMATCH

Step 8 labels the watch head "regression", but this formula is cross-entropy (classification). Two valid designs: **(a) regression** — predict seconds, MSE $(\text{sec_true} - \text{sec_pred})^2$; **(b) weighted classification** — predict a binary "high-quality watch", cross-entropy, with watch time as the per-example *weight*. The formula as written is (b), mislabeled as (a) — **not wrong, just mislabeled** (the author confirms multi-task: some heads regression, some classification). Interview move: name both, note the formula is the weighted-classification one; don't assert one is correct.

L_aux — supporting heads (worked, α=1.0, β=0.5, γ=0.3)

```
clicked=1,    P(click)=0.7      → -log(0.7)    = 0.36
watched 60%, P(completion)=0.5  → (0.6-0.5)2 = 0.01
satisfied=1, P(satisf)=0.4      → -log(0.4)    = 0.92
L_aux = 1.0(0.36) + 0.5(0.01) + 0.3(0.92) = 0.64
```

For clicked=1 the second BCE term $(1-y) \cdot \log(1-p)$ has coefficient $(1-1)=0$, so it vanishes — that's why only `-log(0.7)` remains.

L_position — bias correction (worked)

Logged clicks are contaminated by *where* items were shown — people click the top slot just for being on top. Without correction the model learns "top = good" and mimics the old system.

clicked items, same prediction, base loss 0.22:

slot 1 (top) → weight 0.4 → 0.09 (trust less – position helped)
slot 20 (buried) → weight 1.5 → 0.33 (trust more – real interest)

HOW IS "GOOD SLOT" KNOWN?

It's **measured**, not assigned. A randomization experiment scatters items into random slots; since item quality is then equal across slots, any remaining click-rate difference is the position effect (examination propensity).

$\text{position_weight} = 1/\text{propensity}$ — high-examination slots get small weights. A great video proves itself through clicks in *hard* positions, so it isn't suppressed; the credit just comes from where a click is informative.

Every weight here — watch_time_weight , position_weight , α , β , γ , δ — is a **scalar** (one number scaling a loss), not a matrix. Matrices transform vectors (like Q/K/V); these only dial importance.

Worked example · full conceptual forward pass

One user, 3 history items, scoring one pasta candidate. Dimension 2. All numbers invented to show mechanics; verified by direct computation. Tags: **LEARNED** trained · **COMPUTED** fresh each pass · **SET** human-chosen · **INPUT** data.

Stage 1 — Input processing

	content[LEARNED]	position[LEARNED]	action[LEARNED]	= sum[COMPUTED]	
item0 WATCHED	[1.0, 0.0]	+ [0.0, 0.2]	+ WATCHED[0.0, 0.0]	= [1.0, 0.2]	pasta
item1 LIKED	[0.6, -0.3]	+ [0.0, 0.1]	+ LIKED [0.3, 0.3]	= [0.9, 0.1]	knife
item2 SKIPPED	[0.4, 1.2]	+ [0.0, 0.1]	+ SKIPPED[-0.3,-0.3]	= [0.1, 1.0]	makeup

H = [1.0,0.2 ; 0.9,0.1 ; 0.1,1.0] candidate c = [1.0, 0.3]

Note LIKED [0.3,0.3] and SKIPPED [-0.3,-0.3] are distinct and opposite — that's the whole point of action embeddings.

Stage 2 — Self-attention

W_Q=W_K=I, W_V=0.5·I (all LEARNED; set to clean values for legibility)
√d_k = √2 = 1.414 [SET]

scores Q·K^T: scaled (÷1.414): softmax rows → weights:

1.04 0.92 0.30		0.735 0.651 0.212		0.398 0.366 0.236
0.92 0.82 0.19	→	0.651 0.580 0.134	→	0.395 0.368 0.236
0.30 0.19 1.01		0.212 0.134 0.714		0.280 0.259 0.462

H' (weights · V):

pasta'	[0.376, 0.176]	("pasta, in a cooking-heavy history")
knife'	[0.375, 0.176]	
makeup'	[0.279, 0.272]	(still makeup-leaning)

shape 3×2 – same as input, so blocks can stack

Stage 3 — Cross-attention (candidate queries H')

q = c·W_Q = [1.0, 0.3]; keys/values from H'

scores: ·pasta'=0.428 ·knife'=0.428 ·makeup'=0.361
÷√2: 0.303 0.303 0.255
softmax: [0.339, 0.339, 0.323] (cooking 0.68 vs makeup 0.32)

U = weighted sum of values = [0.172, 0.104] ← candidate-conditioned (UNCACHEABLE)

Stage 4 — Multi-task heads (linear w·U + b, then squash)

head	w[LEARNED]	b	logit=w·U+b	output
click	[2.0,1.0]	0.0	+0.448	sigmoid → 0.610
like	[1.5,1.5]	0.0	+0.414	sigmoid → 0.602
see less	[1.0,1.0]	-2.5	-2.224	sigmoid → 0.098
completion	[0.8,0.8]	0.2	+0.420	linear → 0.420

Stage 5 — Value model (W's are SET)

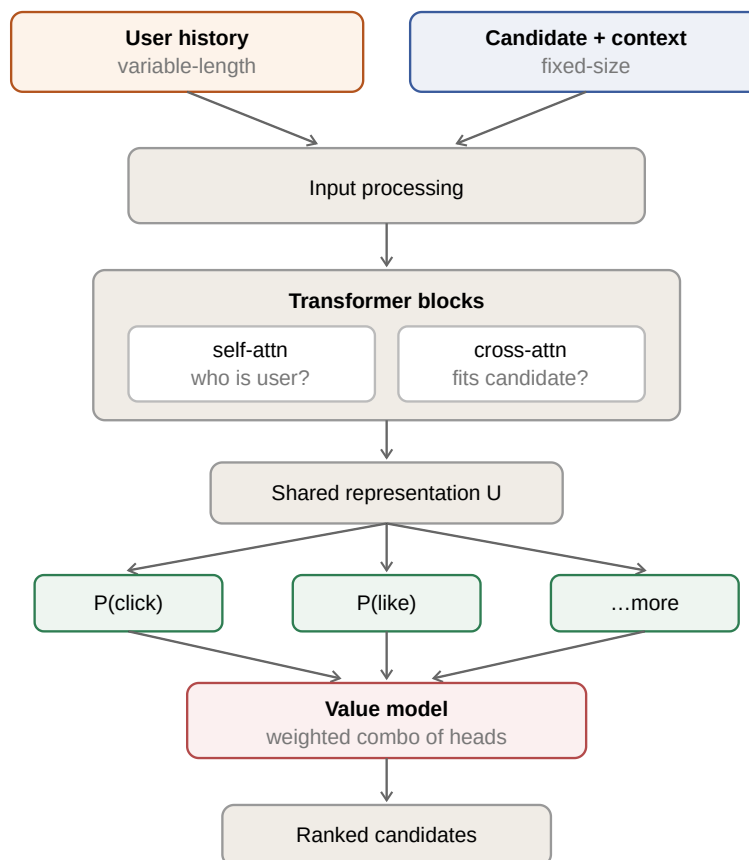
```
W_click=1.0 W_like=2.0 W_see_less=3.0 W_complete=1.0
Value = 1.0(0.610) + 2.0(0.602) - 3.0(0.098) + 1.0(0.420) = 1.94
→ score the other ~99 candidates, sort, return top
```

Stage 6 — Training adjusts this

```
high-quality watch (label=1)[INPUT], watch_time_weight=2.0, watch head predicted 0.58:
L_engage = -1 · log(0.58) · 2.0 = 1.09
→ gradient descent updates all LEARNED (embeddings, both attn W-sets, head weights)
→ leaves SET (value weights, √d_k) and COMPUTED activations untouched
```

WHAT'S LEARNED VS. NOT

The model learns the **machinery** (W matrices, embedding tables, head weights), not the **activations** (scores, attention weights, H', U, outputs). Same machinery on a different user → different computed values. The value-model weights stay **SET** — the seam where the business keeps a hand on the objective without retraining.



The conceptual architecture — the version to whiteboard first.

Part B · Production reality (TransAct)

Step 11 · Where the conceptual design breaks

The conceptual design's strength — the candidate fused deep inside via cross-attention — is its fatal flaw at scale. Three problems:

Problem	Why
Recompute ×100	The candidate enters <i>inside</i> the transformer, so scoring 100 candidates reruns the whole transformer 100× — including the expensive history processing, which is identical every time.
Nothing cacheable	The user representation is <code>f(history, candidate)</code> — a different vector per candidate (the <code>[0.172, 0.104]</code> vs other-candidate vectors). No single value to store. Like caching "7×x": depends on x.
Variable-length serving	Raw variable-length sequences through a transformer every request: latency swings with history length, batching is hard.

THE FIX (ONE SENTENCE)

Stop fusing the candidate into the transformer. Use the transformer **only** to compress the variable-length history into one fixed vector — computed **once per request**, cacheable — then bring the candidate in **later**, cheaply, through a separate combination step.

Step 12 · Evolving into TransAct

Three changes to the conceptual architecture:

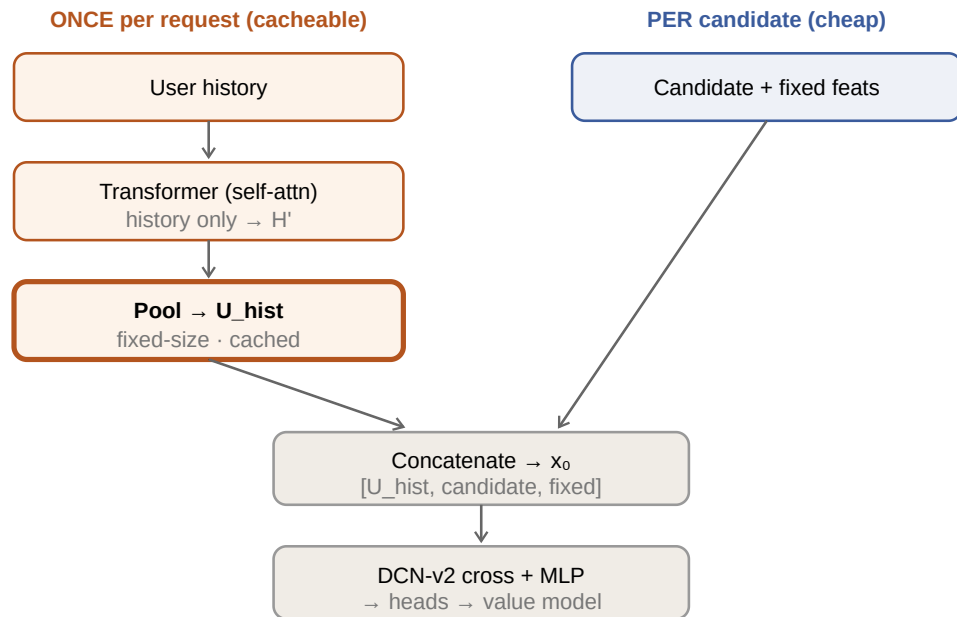
1. **Transformer sees history only** (no candidate) → output H' is candidate-independent, shared across all 100 candidates.
2. **Pooling** collapses the H' matrix to one fixed vector `U_hist` (max- or mean-pool across rows). Because no candidate touched it, it's **cacheable**.
3. **Candidate joins late** — concatenate `U_hist` + candidate + fixed features into `x0`, then a cheap DCN-v2 cross + MLP to the heads.

mean-pool H' :
([0.376+0.375+0.279]/3 , [0.176+0.176+0.272]/3) = [0.343, 0.208]
U_hist = [0.343, 0.208] – fixed-size, candidate-independent, cacheable
(vs conceptual U=[0.172,0.104] which was candidate-dependent)

Problem (Step 11)	How TransAct fixes it
Recompute ×100	Transformer runs once for U_hist; per-candidate work is just concat + small cross net.
Not cacheable	U_hist depends only on history → compute once, reuse/cache.
Variable-length	Pooling → fixed size → everything downstream batches cleanly.

THE TRADE-OFF

We lose candidate-specific history selection inside the transformer (U_hist is a *generic* "this user likes cooking"). We recover some of it in the cheap DCN-v2 cross ($U_hist \times candidate$), but it's coarser than full cross-attention. Deliberate speed-for-power trade.



Left column runs once and caches; right column runs per candidate but is cheap. Moving the candidate to the right column is the entire fix.

What is DCN-v2?

Deep & Cross Network v2 (Google 2020) — learns explicit **feature crosses** (feature×feature products, e.g. "likes cooking" × "is cooking video") efficiently. One cross layer: $x_1 = x_0 \odot (W \cdot x_1 + b) + x_0$ — multiplying by the original x_0 is what creates the products; depth controls interaction order. Run in parallel with a deep MLP. The comment thread: DCN-v2 + FC layers remain SOTA for *fixed-dimensional* data, which is exactly what we have after pooling. (The sources name DCN-v2 but give no formula; the cross-layer equation is from the DCN-v2 paper.)

Step 13 · The full TransAct v2 flow

The production architecture in full — the 9 steps from the comment thread. Runs per candidate, but the expensive history encoding is shared/cacheable.

#	Step
1	Build sequence S — from $\sim 10^4$ history items, take $\sim O(100)$: most-similar + most-recent + recent impressions.
2	Early fusion — concatenate the candidate onto <i>each</i> history item. The candidate enters the sequence cheaply (not via full cross-attention).
3	Add embeddings — interaction type (clicked, pinned), context (feed vs. search), item-age bucket.
4	Causal transformer — 2-layer, causal mask $\rightarrow$ output vectors U.
5	Next-action loss (auxiliary) — some U vectors predict the real next item vs. hard negatives. Training-only; sharpens the encoder; discarded at serving.
6	Pool $\rightarrow$ fixed vector — linear + max-pool over U.
7	Concat fixed features — item, creator, user metadata $\rightarrow x_0$.
8	DCN-v2 cross + MLP $\rightarrow x_{out}$.
9	Heads — linear + softmax per metric $\rightarrow$ value model.

OPEN QUESTION (THE PRACTITIONER FLAGS IT HIMSELF)

Whether the crossing is **early** (one joint shallow cross over U_hist + fixed features) or **late** (more separated). Treat early-vs-late fusion placement as unsettled, not a known answer.

Why causal masking here?

The next-action loss predicts the *future* item, so each position must not see items after it — otherwise prediction is trivial. (The earlier conceptual example used full, non-causal attention.)

Worked example through the 9 steps (dim 2, verified)

```
Step1  history content: pasta[1.0,0.0] knife[0.8,0.0] makeup[0.1,0.9]; candidate=[0.9,0.1]
Step2  early fusion (+0.2xcand to each row)
Step3  + action + age  $\rightarrow$  transformer input X:
        pasta [1.18,0.07]  knife [1.08,0.22]  makeup [0.08,1.07]
Step4  causal self-attention (row i sees only  $\leq i$ ):
        weights          1.00  0      0
                          0.513 0.487 0
                          0.243 0.271 0.486
        U:  pasta[0.590,0.035] knife[0.566,0.072] makeup[0.309,0.299]
Step5  next-action: U_pasta·knife=0.472 vs hard-neg makeup=0.090
        softmax[0.594,0.406]  $\rightarrow$  L_next =  $-\log(0.594) = 0.520$  (aux, training only)
Step6  max-pool U  $\rightarrow$  U_hist = [0.590, 0.299] (fixed, cacheable)
Step7   $x_0$  = concat(U_hist, candidate, creator0.3, topic0.7) = [0.590,0.299,0.9,0.1,0.3,0.7]
Step8  DCN-v2 cross:  $x_0 \cdot W_c = 0.528$ ;  $x_1 = x_0 \odot 0.528 + x_0 = [0.902,0.456,1.375,0.153,0.458,1.070]$ 
Step9  heads: P(click)=0.858, P/watch)=0.833
        value =  $1.0 \cdot P(\text{click}) + 2.0 \cdot P(\text{watch}) = 2.52$ 
```

The two things this makes concrete beyond Step 12: **causal masking** (forced by the next-action objective) and **early fusion** (candidate enters per-item at step 2, not held out entirely). Everything from U_hist onward is fixed-size, so it's cheap per candidate while the encoding is shared. Additive fusion and max-pool are simplifications; real TransAct concatenates and uses learned fusion.

Next-action loss vs. the main loss

Next-action loss		Main loss (Step 10)
Predicts	the user's next item	engagement outcomes
Label from	the sequence itself (self-supervised)	logged reactions
Attaches to	transformer output U (mid-network)	the heads (end)
Purpose	shape the encoder (a means)	the actual goal (an end)
Served?	No — training only	Yes — drives ranking

$$L_{total} = L_{engage} + L_{aux} + L_{position} + \lambda \cdot L_{next_action}$$

<— main (Step 10) —>

<- auxiliary ->

Both run in one training pass, summed; gradients from both flow into the transformer. The engagement loss is what we're after; next-action rides alongside to enrich the representation, then is dropped at serving.

The Reels twist · the UTIS perception layer

Meta's most recent published change to the Reels heavy ranker — a current, differentiating point. It sits **on top of** the MTML heads we just built. Every fact and number below is from the Facebook Reels engineering blog (verified).

THE PROBLEM

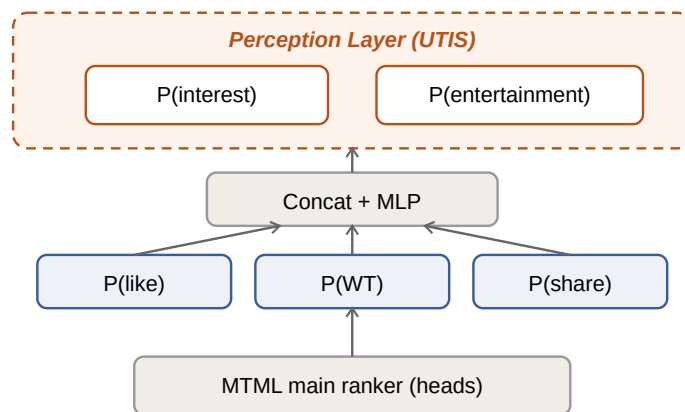
Engagement signals (likes, shares, watch time) are noisy proxies for what users actually want. Models trained only on them optimize short-term engagement but miss **true interest**, which Meta defines as spanning not just topic but *audio, production style, mood, and motivation*. Their existing interest heuristics reached only **48.3% precision** at identifying true interest.

The measurement

A randomly-sampled fraction of daily viewing sessions gets a single-question survey — "To what extent does this video match your interests?" on a 1–5 scale — collecting thousands of in-context responses per day, reweighted to correct for sampling and nonresponse bias.

The model — UTIS (User True Interest Survey)

The main ranker is the MTML model from this section. UTIS is a **lightweight alignment layer** — the "Perception Layer" — trained on the survey responses (binarized to denoise variance) using the main model's **existing predictions as input features**. The trick that makes extremely sparse survey labels usable at scale: the Perception Layer sits on top of the main model's event-prediction heads, with P(like), P(watch-time), P(share) feeding upward through a Concat+MLP into perception heads P(interest) and P(entertainment).



*sparse survey labels generalized via the dense event predictions
the main model already produces*

UTIS Perception Layer sits atop the existing MTML heads.

Where it plugs in (two deployments, both tier-0 retention gains)

Deployment	How
Late-Stage Ranking	UTIS runs in parallel to the LSR model, feeding an extra input into the value formula — high interest gets a modest boost, low interest is demoted. It's one more weighted term in the value model.
Early-Stage (retrieval)	UTIS reconstructs true-interest profiles by aggregating survey data to predict affinity for any user–video pair, re-ranking the profile and sourcing more candidates. User-to-item retrieval sequences are aligned via knowledge-distillation on UTIS-from-LSR predictions as labels.

Reported results (vs. heuristic baseline)

Offline	Online (A/B, >10M users)
accuracy 59.5% → 71.5% precision 48.3% → 63.2% recall 45.4% → 66.1%	+5.4% high survey ratings –6.84% low survey ratings +5.2% total engagement –0.34% integrity violations

REELS FRAMING

"YouTube-style systems optimize logged engagement; Meta found that under-captures true interest, so they run in-feed surveys and train a lightweight perception layer on top of the MTML model's existing heads, then feed its interest score into the value formula at late-stage ranking and use it to source/align candidates at retrieval."

Tricky questions an interviewer will drill on

These are the probing follow-ups that separate a memorized answer from real understanding. Each is a spot where the design has genuine tension — exactly where a strong interviewer pushes.

Q1 · "YOUR WATCH-TIME HEAD IS LABELED REGRESSION, BUT YOUR LOSS IS CROSS-ENTROPY. WHICH IS IT?"

The trap: asserting one confidently. **Answer:** both are valid and they're different models. Regression predicts seconds with MSE; weighted classification predicts a binary "high-quality watch" with cross-entropy, using watch time as the per-example weight. The formula as written is the latter — the "regression" label is a mismatch, not a bug. I'd pick based on whether the value model needs an actual watch-time estimate or just a quality signal. (*Pinterest's TransAct v2 uses weighted cross-entropy per head, so the classification framing is the production norm.*)

Q2 · "THERE'S NO PAIRWISE/LISTWISE RANKING LOSS. DOES THIS EVEN LEARN TO RANK?"

Yes — it's **pointwise** learning-to-rank. Predicting engagement is ranking, because higher predicted engagement → higher sorted position. You don't need an explicit ranking loss when pointwise predictions directly determine the order. Pairwise/listwise are alternatives that sometimes outperform in IR ranking, but pointwise is simpler, parallelizes per-candidate, and is what most production systems use.

Q3 · "WHY CAN'T THE HEAVY RANKER BE CACHED LIKE THE LIGHT RANKER?"

Because cross-attention makes the user representation a **function of the candidate** — a different vector for each of the 100 candidates. There's no single user vector to store (like caching "7×x": depends on x). The light ranker keeps user and item towers *separate*, so each embedding is candidate-independent and cacheable — but that same separation is exactly why it can't use cross features. Caching and cross features are mutually exclusive; the two stages make opposite trade-offs on purpose.

Q4 · "IF YOU DOWN-WEIGHT CLICKS AT THE TOP SLOT, WON'T YOU PUNISH GENUINELY GOOD VIDEOS SHOWN THERE?"

No — the weight scores the **click's reliability given position**, not the video's quality. A good video gets shown across many positions and proves itself through clicks in *hard* (low-examination) slots, where those clicks are weighted heavily. The same low weight that "punishes" a good video at slot 1 also protects you from being fooled by a mediocre one at slot 1 — it has to apply to both, because the weight is about the slot, not the video. Relies on items being shown across varied positions over time (they are).

Q5 · "HOW DO YOU KNOW WHICH SLOT IS A 'GOOD' (HIGH-EXAMINATION) SLOT?"

Measure it, don't assume it. Run a randomization experiment: scatter items into random slots for a small traffic slice. Since item quality is then equal across slots, any remaining click-rate difference is the **position effect** (examination propensity). Set `position_weight = 1/propensity`. The terms to name: examination propensity, inverse-propensity weighting, counterfactual logging.

Q6 · "YOU SAID SUMMED EMBEDDINGS — WHY NOT CONCATENATE CONTENT + POSITION + ACTION?"

Summing keeps every item the same width (`d_model`), which the transformer requires, and is the original-paper convention. Concatenation would widen each row and grow parameters. The trade-off: summing assumes the signals share a space; if you needed them kept fully separable you'd concatenate (and some systems do). Honest note: modern transformers often use RoPE, where position enters *inside* attention rather than being summed at all — so "summed" is the classic scheme, not universal.

Q7 · "IN TRANSACT YOU POOL THE HISTORY INTO ONE VECTOR — DOESN'T THAT LOSE THE CANDIDATE-SPECIFIC MATCHING CROSS-ATTENTION GAVE YOU?"

Yes, partially — and it's a deliberate trade. The pooled `U_hist` is a *generic* user summary, candidate-independent (which is what makes it cacheable and computable once). We recover some candidate-specificity two ways: (1) **early fusion** — the candidate is concatenated onto each history item *before* the transformer, so the encoding isn't fully candidate-blind; (2) the **DCN-v2 cross** later crosses `U_hist` against the candidate. Coarser than full cross-attention, but ~100× cheaper. Speed-for-power, chosen knowingly.

Q8 · "WHY A TRANSFORMER AT ALL? WHY NOT JUST DCN-V2 ON EVERYTHING?"

DCN-v2 + FC layers are SOTA for **fixed-dimensional** features but can't handle a **variable-length, ordered sequence** — they'd need the history pre-flattened, losing order. The transformer's job is exactly the part DCN-v2 can't do: compress variable-length ordered history into a fixed vector. So they're complementary: transformer for the sequence, DCN-v2 for the fixed-feature crossing. Using only DCN-v2 throws away temporal/sequence signal (DLRM's exact weakness).

Q9 · "WHAT'S THE NEXT-ACTION LOSS FOR IF YOU NEVER SERVE NEXT-ITEM PREDICTIONS?"

It's a **self-supervised auxiliary task**: predict the user's actual next item (vs. hard negatives) from the sequence so far. You don't serve it — you include it because forcing the encoder to predict the future makes `U_hist` a sharper representation, which helps the engagement heads you *do* serve. It's summed into training loss ($L_{total} = L_{main} + \lambda \cdot L_{next}$) and discarded at inference. Same multi-task logic as the heads, applied as a pretext. (*Requires causal masking — a position must not see items after it, or next-item prediction is trivial.*)

Q10 · "WHY ARE THE VALUE-MODEL WEIGHTS HAND-SET INSTEAD OF LEARNED?"

Because they encode a **business judgment** (how to trade off engagement vs. likes vs. "see less") that humans want to control without retraining. If learned, the model might tune them to minimize *its* loss in ways that diverge from product goals. Keeping them SET is the seam where the business steers the objective directly. Same reason `watch_time_weight`'s formula is engineer-designed, not learned.

Q11 · "TRAINING DATA COMES FROM YOUR CURRENT RANKER. ISN'T THAT A FEEDBACK LOOP?"

Yes — this is the core data-quality problem. The model only sees what the current system showed, in the positions it showed, so naive training makes it **mimic the current system** rather than learn true preference. Position-bias correction (Q4/Q5) handles the position component; the broader fixes are counterfactual logging and controlled exploration (showing some content the current system wouldn't, to gather unbiased signal). A staff-level answer names the loop explicitly as a systemic risk, not just a per-example fix.

Q12 · "HOW DO YOU HANDLE A BRAND-NEW VIDEO WITH NO ENGAGEMENT HISTORY? (COLD START)"

Lean on **content-based features** — the video's own embedding (visual/audio/text), creator, topic — which need no interaction history. The multi-task heads help: cross-task correlations give a reasonable prediction even when one signal is absent. And **controlled exploration** deliberately shows the new video to a limited, diverse audience to gather initial behavioral signal cheaply. Content features dominate at first and matter less as interaction-based embeddings mature.

30-second interview summary

SAY THIS

"After the light ranker cuts to ~100 candidates, the heavy ranker can finally use the powerful user–item cross features. Model choice goes MLP → DLRM → transformer: an MLP misses interactions and sequence, DLRM handles sparse/dense heterogeneity but treats history as an unordered bag, and a transformer models history as a sequence. It's multi-task — heads for watch time, CTR, like, share, completion, return-visit — which regularizes the model and feeds a value model at re-ranking. It learns to rank pointwise: predict engagement, sort by a value-model score, no explicit ranking loss needed. The loss is a weighted sum of an engagement term, auxiliary terms, and a position-bias term, since training data is logged from the current system. The key production caveat: nobody ships a pure end-to-end transformer at this scale — the standard is TransAct, where the transformer is a sequence encoder producing a fixed-size interaction embedding that's early-fused with the candidate, concatenated with fixed features, and run through DCN-v2 crossing into the heads."

Source coverage

- **Alex Xu PDF** — funnel framing; thin on heavy ranker (scoring = heavier model, more features).
- **Instagram Explore** — MTML second-stage ranker, consumes cross features, value-model formula.
- **Hello Interview** — Bad/Good/Great, transformer architecture, three-part loss, position-bias.
- **Hello Interview comments** — TransAct v2 9-step flow, watch-head regression-vs-classification, pointwise framing, DCN-v2, early-vs-late open question.
- **Reels blog** — UTIS perception layer on top of the MTML heads (included in this guide; all metrics verified).

Deferred to Section 6 (Serving): value-model weight tuning (Bayesian vs. offline); counterfactual logging / inverse-propensity reweighting.

For a 4.3 revisit: naming the candidate-generation triplet loss as "contrastive learning".